Project Proposal & Literature Review

Lightweight Learned Image Compression for Laptop-Class Training

[TODO: Student A] Mohamed Abdelazeem **[TODO: Student C]**

Milestone 1
CSE681 — Digital Image Processing

Abstract

[TODO: Complete this abstract as a team. In 3–5 sentences, state the problem, motivation, planned learned-compression approach, dataset, and JPEG baseline. Mention that Student C has established the initial Kodak/JPEG evaluation pipeline and generated preliminary rate–distortion results.]

1 Problem Statement and Motivation

[TODO: Student A/B: Describe the problem, project scope, motivation, and connection to a course topic. Explain why learned image compression is worth investigating under laptop-class compute constraints.]

2 Related Work

[TODO: Student A/B: Summarize at least four references, including at least two from 2023–2026. Explain how the work informs this project. Keep the bibliography entries in the shared references.bib file.]

Table 1: Key references and their relevance to this project.

Reference	Year	Relevance
(Ballé et al., 2018)	2018	Scale-hyperprior learned image codec.
(He et al., 2022)	2022	Efficient learned image compression architecture.
[TODO: Recent reference]	**[TODO: Year]**	**[TODO: Relevance]**
[TODO: Recent reference]	**[TODO: Year]**	**[TODO: Relevance]**

3 Dataset and Baseline Plan

The Kodak image dataset is used as the common evaluation dataset for the compression methods in this project. It contains 24 test images, each with a resolution of 768×512 pixels. The same original

images will be used to evaluate both the conventional JPEG baseline and the learned compression model. Using an identical test set makes the comparison more consistent.

The JPEG baseline is implemented in Python using the Pillow library. JPEG quality settings are varied to obtain multiple operating points with different file sizes and reconstruction qualities. For every compressed image, the actual output file size is recorded. The compressed file is then decoded, and the reconstructed image is compared with its original Kodak image.

The current preliminary plots contain five averaged JPEG operating points. The exact quality settings associated with each point should be taken from the experiment CSV rather than inferred from the plotted coordinates. The evaluation script can also be run for the full planned quality list, if required by the team protocol.

The JPEG results establish a conventional compression baseline. Once the learned codec is available, it will be evaluated on the same Kodak images using the same metric definitions.

Dataset source: https://r0k.us/graphics/kodak/kodak/

4 Proposed Method

[TODO: Student B: Describe the reduced-width hyperprior codec, planned width or latent-channel settings, and include an architecture figure or pseudocode. Student A: add any relevant data/training details.]

We plan to use a reduced-width learned image compression model based on a hyperprior architecture. The encoder transforms an image into a smaller latent representation, which is quantized for compression. A hyperprior helps estimate the latent values so they can be encoded efficiently. The decoder uses the encoded representation to reconstruct the image. This design follows the general approach of learned hyperprior codecs (Ballé et al., 2018), while drawing on later work on efficient learned compression (He et al., 2022).

The model will use fewer channels than a standard configuration to keep training manageable with our available compute. The exact channel widths and training settings will be selected during implementation and reported with the experiments. We will compare reconstructed images against the originals using the evaluation pipeline described above. Where possible, the bitrate will be calculated from the actual encoded bitstream rather than an estimated target rate.

Figure 1 shows the planned codec architecture. We will use a reduced-width model to make training more manageable. The exact channel widths and training settings will be chosen during implementation. We will first try training on a laptop and use Google Colab if the laptop's runtime or memory is not sufficient.

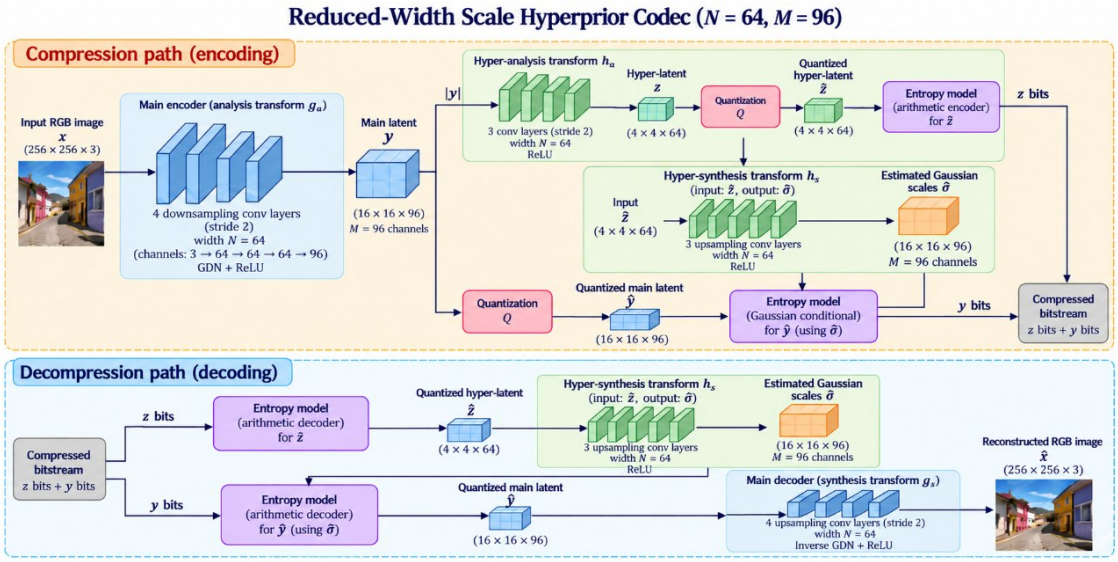


Figure 1: Planned architecture for the learned image-compression codec.

5 Evaluation Plan

The evaluation is designed to compare compression rate, reconstructed image quality, and decoding performance. Three primary image compression metrics are used.

- **Bits per pixel (BPP):** BPP measures the average number of encoded bits per image pixel. For a compressed file with size measured in bytes,

$$\text{BPP} = \frac{8 \times \text{compressed file size (bytes)}}{\text{image width} \times \text{image height}}.$$

  A lower BPP indicates a smaller encoded representation.

- **Peak signal-to-noise ratio (PSNR):** PSNR measures pixel-level reconstruction error and is reported in decibels (dB). For images represented with a maximum possible pixel value MAX, it is calculated from the mean squared error:

$$\text{PSNR} = 10 \log_{10} \left(\frac{MAX^2}{\text{MSE}} \right).$$

  Higher PSNR generally indicates lower reconstruction error.

- **Multi-scale structural similarity (MS-SSIM):** MS-SSIM compares structural information at multiple image scales. Its commonly used range is 0 to 1, with values closer to 1 indicating greater structural similarity.

For each JPEG operating point, BPP, PSNR, and MS-SSIM are calculated for the Kodak images. The per-image measurements are retained, and the average metric values are used to produce the summary plots. BPP is calculated from the actual compressed file size, rather than from an estimated entropy or target bitrate.

Two rate–distortion plots are generated:

- Average PSNR (dB) versus BPP.
- Average MS-SSIM versus BPP.

The same evaluation procedure will be applied to the learned codec. The comparison will use the same original Kodak images, image dimensions, color handling, and metric definitions. The final report will include the individual measurements where appropriate, averages, and rate–distortion curves. Decoding time will also be measured on the same hardware environment for both methods. Timing will use warm-up runs and repeated measurements; file reading time should be excluded where practical so that the measurement focuses on decoding.

5.1 Preliminary JPEG Baseline Results

The first evaluation experiment establishes the JPEG baseline on the Kodak dataset. The current plots show five operating points, with average BPP values spanning approximately 0.33 to 2.35 bits per pixel. The values below are approximate readings from the plotted figures; the experiment CSV should be treated as the authoritative source for exact numerical results.

Figure 2 shows the average PSNR as a function of BPP. The plotted PSNR increases from approximately 26.7 dB at the lowest bitrate to approximately 38 dB at the highest bitrate. This illustrates the expected rate–distortion trade-off: allocating more bits generally improves reconstruction quality.

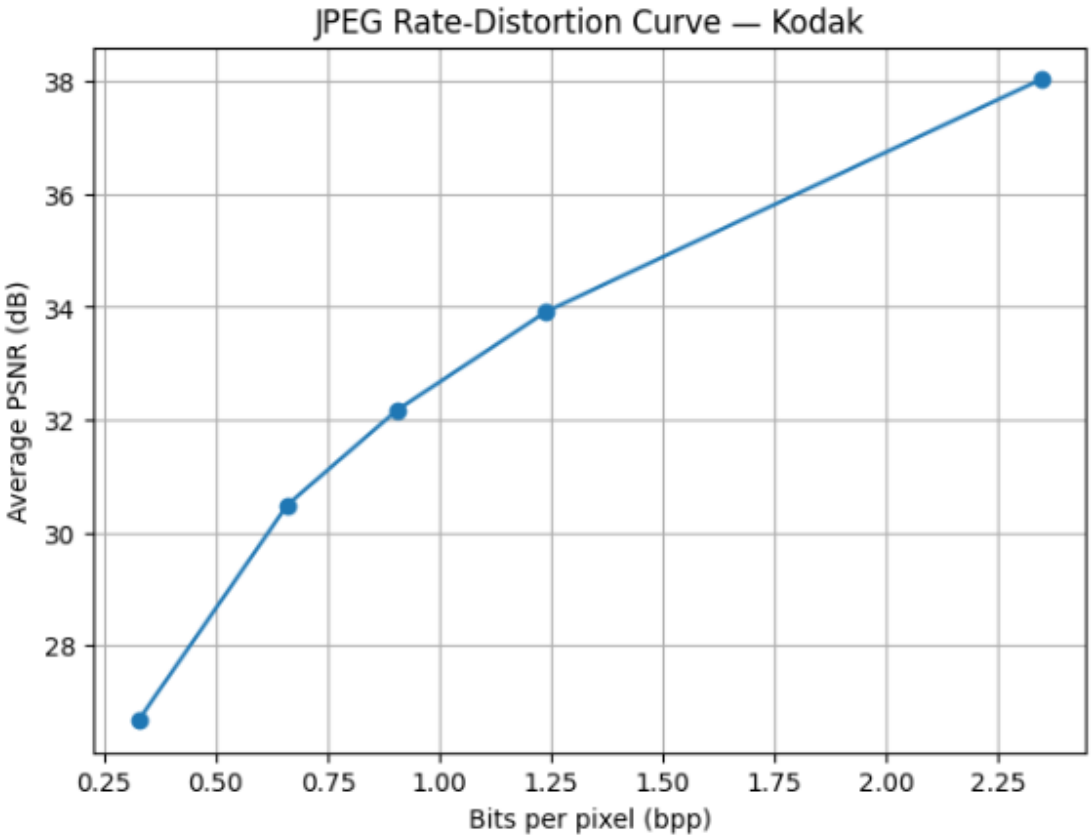


Figure 2: JPEG rate–distortion curve using average PSNR on the Kodak dataset.

Figure 3 shows the average MS-SSIM at the same plotted operating points. The value increases from approximately 0.894 to approximately 0.993 as BPP increases. The curve also suggests that the improvement in MS-SSIM becomes smaller at higher bitrates, which is consistent with diminishing perceptual-quality gains as more bits are allocated.

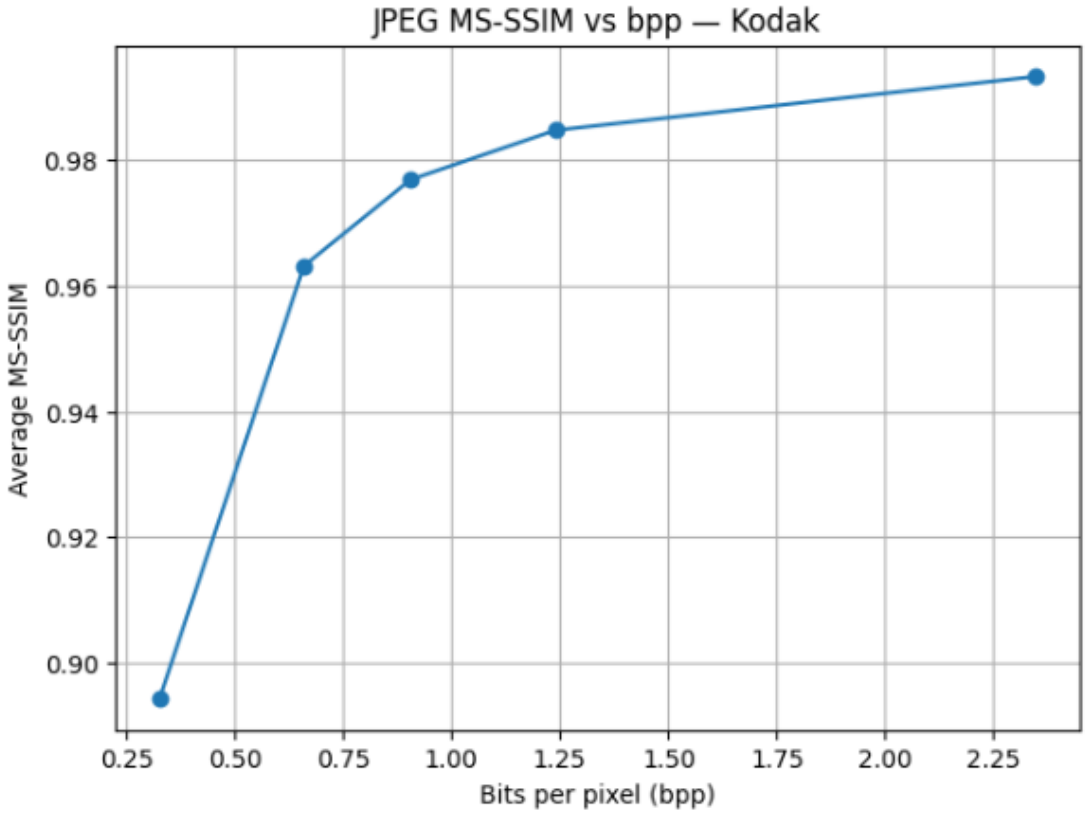


Figure 3: Average JPEG MS-SSIM versus BPP on the Kodak dataset.

These results are preliminary baseline measurements, not a comparison against the learned codec. The learned-codec results will be added after the trained model and its actual bitstream encoder/decoder are available.

5.2 Evaluation Implementation

The evaluation pipeline is implemented in Python in Google Colab. The main libraries used are:

- **Pillow (PIL):** JPEG compression and image decoding.
- **NumPy:** Image-array processing and numerical operations.
- **Pandas:** Organizing and exporting per-image and averaged experimental results to CSV.
- **Matplotlib:** Generating the rate–distortion plots.
- **scikit-image:** PSNR calculation.
- **pytorch-msssim:** MS-SSIM calculation.

- **PyTorch:** Tensor handling required by the MS-SSIM implementation.

The pipeline processes the Kodak images at each JPEG quality setting, saves the compressed outputs, records their file sizes, decodes them, and calculates the quality metrics against the corresponding original images. The measurements are saved in CSV format so that plots and summary tables can be regenerated.

The metric-calculation functions are kept separate from the codec implementation. This allows the same evaluation functions to be reused for the learned compression model and helps maintain a consistent comparison between JPEG and the learned codec.

6 Feasibility and Compute Plan

[TODO: Student B: State the compute tier, training setup, expected experiment time, and result of the one-training-step feasibility test. Student A/B: describe training hardware and memory constraints.]

We will begin by trying to train the model on a laptop. If the required runtime or memory is not practical, we will use Google Colab when available. The available hardware may vary, so we will record the device used for each experiment.

Before starting a longer run, we will perform a short feasibility test using one training step. We will record whether the step completes, its runtime, and any memory or configuration limits. This test has not yet been completed, so we cannot provide a measured runtime or training estimate at this stage. We will use the test results to choose the model width, image crop size, and batch size. If needed, we will reduce these settings to make training feasible.

6.1 Minimum Viable Deliverable

[TODO: As a team, state the smallest working experiment that produces a valid comparison with JPEG. At minimum, it should include a working learned encoder/decoder, actual encoded bitstream size, and evaluation on the same Kodak images.]

6.2 Stretch Goal

[TODO: As a team, state what will be attempted if the MVP is on schedule by Week 5–6, such as additional model-width settings, another evaluation dataset, or comparison with published operating points.]

7 Risks and Mitigations

- **[TODO: Student A/B: Training may exceed available laptop or Colab compute. State the reduced-width or reduced-crop fallback.]**
- **[TODO: Student B/C: The learned codec may not produce a valid bitstream early enough. State how the team will preserve a working end-to-end evaluation path and report limitations.]**
- **[TODO: Student C: Differences in image preprocessing or metric definitions could make the comparison unfair. Use the same Kodak originals, dimensions, color conventions, and metric implementation for both codecs, and retain per-image results.]**

8 Work Breakdown

Table 2: Task allocation by milestone and team member.

Milestone	Student A	Student B	Student C
M1 (Week 3)	**[TODO: Dataset preparation and JPEG baseline tasks]**	Review hyperprior design; choose a reduced-width model and run a one-step feasibility test.	Prepare Kodak evaluation dataset; implement BPP, PSNR, and MS-SSIM calculations; establish the JPEG baseline methodology and generate preliminary plots.
M2 (Week 6)	**[TODO: Training/data pipeline progress]**	Implement the training and encode/decode pipeline; train the model and verify that it produces a valid bitstream.	Complete and validate JPEG experiments; organize per-image and averaged results; generate and document preliminary rate–distortion curves.
M3 (Week 9)	**[TODO: Final data/training contribution]**	Finalize the trained model and bitstream encoder/decoder; document the model and training settings and support the final evaluation.	Evaluate the learned codec on Kodak; compare it against JPEG using rate–distortion curves; measure decoding time; prepare final result tables and qualitative comparisons.

9 Student C Deliverables and Current Status

The following evaluation tasks have been completed for the initial JPEG baseline:

- Downloaded and prepared the 24-image Kodak evaluation dataset.
- Implemented BPP calculation using the actual compressed JPEG file size and image dimensions.
- Implemented PSNR and MS-SSIM calculation functions.
- Ran JPEG compression experiments and collected preliminary measurements.
- Generated average PSNR–BPP and MS-SSIM–BPP plots.

The remaining evaluation tasks are to run the same pipeline on the learned codec, verify that BPP is based on the actual encoded bitstream, measure decoding time under a consistent protocol, and produce the final JPEG-versus-learned-codec comparison.

References

Johannes Ballé, David Minnen, Saurabh Singh, Sung Jin Hwang, and Nick Johnston. Variational image compression with a scale hyperprior. In *International Conference on Learning Representations*, 2018.

Dailan He, Ziming Yang, Weiming Peng, Rui Wang, Hongwei Ma, Hong Cheng, et al. ELIC: Efficient learned image compression with unevenly grouped space-channel contextual adaptive coding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.